p1040R0
std::embed

Published Proposal, 7 May 2018

Author:
JeanHeyd Meneide

Audience:
EWG, LEWG, SG15

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/ThePhD/embed/blob/master/papers/source/P1040 - embed.bs

---

Abstract

Accessing program-external resources at compile-time and making them available to the developer.

Table of Contents

This paper introduces a function `std::embed` in the `<embed>` header for pulling resources at compile-time into your program and optionally guaranteeing that they are stored in the resulting program in an implementation-defined manner.

§ 1. Motivation

Every C and C++ programmer -- at some point -- attempts to `#include` large chunks of non-C++ source into their code. Of course, `#include` expects the format of the data to be source code, and thusly the program fails with spectacular lexer errors. Tools such as `xxd -i` have been purposed to help users deal with this. Many developers also hand-wrap their files in (raw) string literals, or similar to massage their data -- binary or not -- into a conforming representation that can be parsed at source code. This very quickly becomes untenable, for many reasons:

1. If the data is updated, a pre-build step needs to be added (or, worse, the programmer commits to manual updates).

   a. The tools that generate these source-ready formats are not part of any compiler, and thusly inhibit portability of such code.

b. Requiring manual updates and tools drastically hurts the ability of many domains (embedding static shader code, large firmware chunks, unchanged image data, et cetera) to use C++ effectively in data-rich/data-driven applications which require many assets to be baked at build-time.

c. It makes C++ even harder to teach and use across platforms for multiple use cases.

2. Lexing and Parsing data-as-source-code adds an enormous overhead to actually reading and making that data available.

   a. Binary data as C(++) arrays provide the overhead of having to comma-delimit every single byte present, it also requires that the compiler verify every entry in that array is a valid literal or entry according to the C++ parser.

   b. This scales poorly with larger files, and build times suffer for any non-trivial binary file, especially when it scales into Megabytes in size (e.g., firmware and similar).

The request for some form of `#include_string` or similar dates back quite a long time, with one of the oldest stack overflow questions asked-and-answered about it dating back nearly 10 years. Predating even that is a plethora of mailing list posts and forum posts asking how to get script code and other things that are not likely to change into the binary.

This paper proposes `<embed>` to make this process much more efficient, portable, and streamlined.

§ 2. Scope and Impact

`embedded embed( string_view resource_identifier, embed_options options = embed_options::none )` is an extension to the language proposed entirely as a library construct. The goal is to have it implemented with compiler intrinsics or other suitable mechanisms. It does not affect the language and the header is entirely its own.

§ 3. Design Decisions

`<embed>` avoids using the preprocessor or defining new string syntax, preferring the use of a free function in the `std` namespace and some associated utility flags and structures. `<embed>`'s design is derived heavily from community feedback plus the rejection of the prior art up to this point, as well as the community needs demonstrated by existing practice and their pit falls.

§ 3.1. Current Practice

There are a few cross-platform (and not-so-cross-platform) paths for getting data into your executable.

§ 3.1.1. Pre-processors and Massages

Many developers -- if their data is small enough or if it is in some textual representation -- forego things like proper syntax highlight and tool help by prepending and appending (raw) string literal syntax to their files. This happens often in the case of people who have not yet taken the "add a build step" mantra to heart.

Other developers use preprocessors for data that can't be easily hacked into a C++ source-code appropriate state. The most popular one is `xxd -i my_data.bin`, which outputs an array in a file which developers then include. The name of the array is the named after the input file (e.g., `char my_data[] = { ... }`).

Others still use python or other small scripting languages as part of their build process, outputting data in the exact C++ format that they like.

§ 3.1.2. `ld`, resource files, and other vendor-specific tools

Resource files and other "link time" or post-processing measures have one benefit over the previous method: they are fast to perform in terms of compilation time. However, they come with the cost of losing the ability to perform compile-time inspection and manipulation.

For example, one can embed data using `ld -r -b binary -o my_obj.o path/to/data.bin`. It is then your responsibility to create a source file with, e.g.:

```
extern const unsigned char _binary_data_bin_start[];
extern const unsigned char _binary_data_bin_end[];
size_t const size = _binary_data_bin_end - _binary_data_bin_start;
```

(Thank you to Arvid Gerstmann for demonstrating the snippet).

Because these declarations are `extern`, the values in the array cannot be accessed at compilation/translation-time. While this is not a problem for some, it is for others and -- of course -- all of these techniques are compiler and vendor specific.

§ 3.2. Prior Art

There has been a lot of discussion over the years in many arenas, from Stack Overflow to mailing lists to meetings with the Committee itself. The latest advancements that had been brought to WG21's attention was [[p0373r0 | p0373r0 - File String Literals]]. It proposed the syntax `F"my_file.txt"` and `bF"my_file.txt"`, with a few other amenities, to load files at compilation time. The following is an analysis of the previous proposal.

§ 3.2.1. Literal-Based, constexpr

A user could reasonably assign (or want to assign) the resulting array to a `constexpr` variable as its expected to be handled like most other string literals. This allowed some degree of compile-time reflection. It is entirely helpful that such file contents be assigned to constexpr: e.g., string literals of JSON being loaded at compile time to be parsed by Ben Deane and Jason Turner in their CppCon 2017 talk, constexpr All The Things.

§ 3.2.2. Literal-Based, Null Terminated (?)

It is unclear whether the resulting array of characters or bytes was to be null terminated. The usage and expression imply that it will be, due to its string-like appearance. However, is adding an additional null terminator fitting for desired usage? From the existing tools and practice (e.g., `xxd -i` or linking a data-dumped object file), the answer is no: but the syntax makes the answer seem like a "yes".

§ 3.2.3. Encoding

Because the proposal used a string literal, several questions came up as to the actual encoding of the returned information. The author gave both `bF"my_file.txt"` and `F"my_file.txt"` to separate binary versus string-based arrays of returns. Not only did this conflate issues with expectations in the previous section, it also became a heavily contested discussion on both the mailing list group discussion of the original proposal and in the paper itself. This is likely one of the biggest pitfalls between separating "binary" data from "string" data: imbuing an object with string-like properties at translation time provide for all the same hairy questions around source/execution character set and the contents of a literal.

3.3. Design Goals

Because of the aforementioned reasons, it seems more prudent to take a "compiler intrinsic"/"magic function" approach. The function takes the form:

```
...
namespace std {
  embedded embed( string_view resource_identifier, embed_options options =
}
```

`resource_identifier` is processed in an implementation-defined manner to pull resources into C++. The most obvious source will be the file system, with the intention of having this evaluated at compile-time.

3.3.1. Implementation Defined

Calls such as `std::embed( "my_file.txt" );` and `std::embed( "data.dll" );` are meant to be evaluated in a `constexpr` context, where the behavior is implementation-defined. The effect is unspecified behavior when evaluated in a non-constexpr context (with the expectation that the implementation to provide a failing diagnostic in these cases). This is similar to how include paths work, albeit `#include` interacts with the programmer through the preprocessor. There is, however, precedent for specifying library features that are implemented only through compile-time compiler intrinsics (`type_traits`, `source_location`, and similar utilities).

3.3.2. Binary Only

Creating two separate forms or options for loading data that is meant to be a "string" always fuels controversy and debate about what the resulting contents should be. The problem is sidestepped entirely by demanding that the resource loaded by `std::embed` represents the bytes exactly as they come from the resource, modulo any options passed to `std::embed`. This prevents encoding confusion, conversion issues, and other pitfalls related to trying to match the user's idea of "string" data or non-binary formats. Data is received exactly as it is on the machine, whether it is a supposed text file or otherwise. `std::embed( "my_text_file.txt" )` and `std::embed( "my_binary_file.bin" )` behave exactly the same as far as their treatment of the resource.

3.3.3. Opt-in, Optional Null Termination

With the Binary Only stipulation, some users will feel left out as there are many system calls, source processing APIs, and other interfaces which require a null terminated sequence. Therefore, one of the options of `std::embed_options` is `std::embed_options::null_terminated`. If this option is specified, then the data is null terminated.

3.3.4. Options

Similar to the above, `std::embed`'s base behavior can be extended using options. This allows for simple needs that come up in the future or that are missed due to oversight to be corrected for. Currently, the only option flag is for `std::embed_options::null_terminated`, but there might be future flags that do things like e.g. force the data to be baked into a specific section of the binary. Having such flags be standardized means that compiler vendors would have to agree about said flags and their specification before shipping: this is *not the place for vendors to place their own implementation-specific flags* (the goal is to reduce fragmentation, not encourage it).

3.3.5. Constexpr Compatibility

The entire implementation must be usable in a `constexpr` context. It is not just for the purposes of processing the data at compile time, but because it matches existing implementations that store strings and huge array literals in a that are placed into a variable via `#include`. These variables can be `constexpr`: to not have a constexpr implementation is to leave many of the programmers who utilize this behavior out in the cold.

4. Help Requested

The author of this proposal is extremely new to writing standardese. While the author has read other papers and consumed the standard, there is a definite need for help and any guidance and direction is certainly welcome. The author expects that this paper may undergo several revisions and undertake quite a few moments of "bikeshedding".

4.1. Feeling Underrepresented?

The author has consulted dozens of C++ users in each of the Text Processing, Video Game, Financial, Embedded and Desktop Application development subspaces. The author has also queried the opinions

of Academia. The author feels this paper adequately covers many use cases, existing practice and prior art. If there is a use case or a problem not being adequately addressed by this proposal, the author encourages anyone and everyone to reach out to have their voice heard.

4.2. Bikeshedding

4.2.1. Open Questions

There are some open questions about `std::embedded`. `std::embedded` strives to be more like a `view` rather than an owning container, similar to the up and coming p0546r2 - Span.

Should `std::embedded` just be replaced with a fully constexpr `std::span`? It seems like it would be a very good fit, and avoiding "type spam" would help out immensely.

4.2.2. Answered Questions

Question: *Pulling in large source files on every compile might be expensive?*

Answer: We fully expect implementations to employ techniques already in use for compilation dependency tracking to ensure this is not a problem.

Question: *What is the lookup scheme for files and other resources?*

Answer: Implementation defined. We expect compilers to expose an option similar to `--embed-paths=...`, `/EMBEDPATH:...`, or whatever tickles the implementer's fancy.

5. Header Overview

`<embed>` Overview

```
namespace std {
  enum class embed_options {
    none = 0,
    null_terminated = 1
  };

  // bit-flag manipulations
  constexpr embed_options operator| (embed_options left, embed_options righ
  constexpr embed_options operator& (embed_options left, embed_options righ
  constexpr embed_options operator^ (embed_options left, embed_options righ
  constexpr embed_options& operator|= (embed_options& left, embed_options r
  constexpr embed_options& operator&= (embed_options& left, embed_options r
  constexpr embed_options& operator^= (embed_options& left, embed_options r
} // namespace std
```

1. `std::embed_options` is the enumeration that specifies additional transformations that the implementation should do to modify the data made available through `std::embedded`.
2. The operators are provided for ease of use to combine and otherwise modify flags, now and into the future.

```
#include <string_view> // for std::string_view
#include <cstddef> // for std::byte, std::size_t, std::ptrdiff_t...

namespace std {
  struct embedded {
  public:
    // standard type definitions
    typedef std::byte&                              reference;
    typedef const std::byte&                        const_reference;
    typedef const std::byte* const                  iterator;
    typedef const std::byte* const                  const_iterator;
    typedef size_t                                  size_type;
    typedef ptrdiff_t                               difference_type;
    typedef std::byte                               value_type;
    typedef std::reverse_iterator<iterator>         reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

    // constructors
    constexpr embedded() noexcept;
    constexpr embedded(const std::byte* const, std::size_t) noexcept;
    constexpr embedded(const embedded&) noexcept = default;
    constexpr embedded(embedded&&) noexcept = default;

    // capacity requirements
    constexpr size_type size() const noexcept;
    constexpr size_type capacity() const noexcept;
    constexpr bool empty() const noexcept;

    // iterator requirements
    constexpr iterator begin() const noexcept;
    constexpr iterator end() const noexcept;
    constexpr iterator cbegin() const noexcept;
    constexpr iterator cend() const noexcept;

    constexpr iterator rbegin() const noexcept;
    constexpr iterator rend() const noexcept;
    constexpr iterator rcbegin() const noexcept;
    constexpr iterator rcend() const noexcept;

    // element access
    constexpr const std::byte* const data() const noexcept;
    constexpr const std::byte& operator[](size_type) const noexcept;
```

```
  };
} // namespace std
```

1. `std::embedded` will provide a view to *contiguous* storage.
2. If the option `std::embed_options::null_terminated` is specified, then the expressions `*v.end()` and `*(v.data() + v.size())` will evaluate to and compare equal to the value `0`.

```
namespace std {
  constexpr embedded embed( std::string_view resource_identifier, embed_opt
} // namespace std
```

1. The implementation defines what strings it accepts for `resource_identifier`. [Note -- It is the hope that compiler vendors will provide a mechanism similar to include paths for finding things on the local filesystem. -- End Note]
2. If this function is called at runtime, then the behavior is unspecified.

§ 6. Acknowledgements

A big thank you to Andrew Tomazos for replying to the author's e-mails about the prior art. Thank you to Arthur O'Dwyer for providing the author with incredible insight into the Committee's previous process for how they interpreted the Prior Art.

A special thank you to Agustín Bergé for encouraging the author to talk to the creator of the Prior Art and getting started on this. Thank you to Tom Honermann for direction and insight on how to write a paper and apply for a proposal.

Thank you to Lilly (Cpplang Slack, @lillypad) for the valuable bikeshed and hole-poking in original designs, alongside Ben Craig who very thoroughly explained his woes when trying to embed large firmware images into a C++ program for deployment into production.

For all your hard work, I hope I can get this into C++. It would be my distinct honor to make all of your lives easier and better with the programming language we work in and love. ♥

§ References

§ Informative References

[CONSTEXPR-ALL-THE-THINGS]

Ben Deane; Jason Turner. constexpr All The Things: CppCon 2017. September 25th, 2017.

[P0546R2]

H. Carter Edwards, Daniel Sunderland. Span - foundation for the future. URL: https://wg21.link/p0546r2